

Verifying ABA with Leslie

Gaspard Reghem

June 22, 2026

Abstract

Our main objective is the verification of an implementation of Asynchronous Byzantine Agreement. The proof of correctness consist in building a simulation relation between the implementation and an implementation running in an idealised setting. The simulation ensures that the correctness of the idealised implementation implies the correctness of the original one. However, this implementation is complex, thus, building the simulation in one go is intractable. Fortunately, the implementation is designed as the interlocking of multiple sub-protocols (GBCA, WCC, RSD, Gather, AVSS, BRB). This architecture allows us to decompose the construction of the overall simulation into smaller steps. Each implementation of sub-protocols will be simulated by an idealised version. Thus, when building the simulation for a protocol using sub-protocols, the sub-implementations will be replaced by their idealised versions; making the construction easier.

1 Preliminaries

1.1 Traditional Framework

Definition 1 (Probabilistic Labelled Transition System). A probabilistic labelled transition system (PLTS) is a tuple $\mathcal{P} := (Q, q_*, L, T)$ where Q is a (countable) set of states, $q_* \in Q$ is the initial state, L is a (countable) set of labels and $T \subseteq Q \times (L \cup \{\tau\}) \times \mathcal{D}(Q)$ is a set of probabilistic transitions.

A partial execution of $\mathcal{P}$ is a sequence $q_0.l_0.q_1.l_1 \dots \in (Q.L)^*.Q \cup (Q.L)^\omega$ such that, for every $n \in \mathbb{N}$, there exists $\mu_n \in \mathcal{D}(Q)$ with $(q_n, l_n, \mu_n) \in T$ and $q_{n+1} \in \bar{\mu}_n$. The set of finite executions is denoted $\mathcal{E}^*$. A partial execution is called an execution if $q_0 = q_*$. The trace of a partial execution is its projection onto L . The set of traces of $\mathcal{P}$ is denoted $\mathcal{T}$. A scheduler s is a function from $\mathcal{E}^*$ to $\mathcal{D}(T \cup \{\perp\})$ such that, for all $e \in \mathcal{E}^*$ and $t \in s(e)$, $t = \perp$ or $src(t) = last(e)$. The set of schedulers is denoted Sch .

Definition 2 (Partial Probabilistic Execution). Given $\mu \in \mathcal{D}(Q)$ and $s \in Sch$, one can define the partial probabilistic execution induced by (μ, s) as the function $\mathbf{e}_{\mu,s}$ from $\mathcal{E}^*$ to $[0, 1]$ such that if $e \in Q$ then $\mathbf{e}_{\mu,s}(e) = \mu(e)$; otherwise, $e = e'.l.q$ and $\mathbf{e}_{\mu,s}(e) = \mathbf{e}_{\mu,s}(e') \cdot \sum_{t \in T | lbl(t)=l} s(e')(t) \cdot out(t)(q)$.

A partial probabilistic execution is called a probabilistic execution if it is induced by a pair (μ, s) such that $\mu = \delta_{q_*}$. Partial probabilistic executions induce a measure on the σ -field induced by cones of partial executions, and the trace function is measurable from the σ -field induced by cones of partial executions to the one induced by cones of traces. Thus, $\mathbf{t}_{\mu,s} = tr^{-1} \circ \mathbf{e}_{\mu,s}$ is well-defined and called a trace distribution. The set of trace distributions achievable by $\mathcal{P}$ is defined as those induced by a probabilistic execution of $\mathcal{P}$ and denoted $tdist(\mathcal{P})$. The trace distribution pre-order $\mathcal{P} \subseteq^t \mathcal{P}'$ is defined by $tdist(\mathcal{P}) \subseteq tdist(\mathcal{P}')$.

Definition 3 (Parallel Composition of PLTS). Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$ and $S \subseteq L \cup L'$, the composed PLTS $\mathcal{P} \parallel_S \mathcal{P}'$ is defined as $(Q \times Q', (q_*, q'_*), L \cup L', T \parallel_S T')$ where $((q, q'), l, \mu'') \in T \parallel_S T'$ if and only if

- $l \in S$ and there exist $(q, l, \mu) \in T$ and $(q', l, \mu') \in T'$ such that $\mu \otimes \mu' = \mu''$; or
- $l \notin S$ and there exists $(q, l, \mu) \in T$ such that $\mu \otimes \delta_{q'} = \mu''$; or
- $l \notin S$ and there exists $(q', l, \mu') \in T'$ such that $\delta_q \otimes \mu' = \mu''$.

The trace distribution pre-order is not stable under $\parallel_S$: $\mathcal{P} \subseteq^t \mathcal{P}'$ does not imply $\mathcal{P} \parallel_S \mathcal{P}'' \subseteq^t \mathcal{P}' \parallel_S \mathcal{P}''$ [Seg95]. Thus, the largest pre-order included in $\subseteq^t$ that is stable under $\parallel_S$ is called the trace distribution pre-congruence and is denoted $\sqsubseteq^t$.

Given $R \subseteq A \times \mathcal{D}(B)$, the probabilistic lifting of R , denoted $\mathcal{D}(R)$, is defined by $(\mu_A, \mu_B) \in \mathcal{D}(R)$ if and only if there exists $w : R \rightarrow [0, 1]$ such that $\mu_A = a \mapsto \sum_{\mu \in \mathcal{D}(B)} w(a, \mu)$ and $\mu_B = \sum_{(a, \mu) \in R} w(a, \mu) \cdot \mu$. If $s \in Sch$ terminates almost surely from μ (i.e. $\sum_{e \in \mathcal{E}^*} \mathbf{e}_{\mu,s}(e) \cdot s(e)(\perp) = 1$) then $\mathbf{q}_{\mu,s} = last^{-1} \circ \mathbf{e}_{\mu,s}$ is a well-defined probability distribution over states. We denote $\mu \xRightarrow{l} \mu'$ if and only if there exists $s \in Sch$ such that $\delta_l = \mathbf{t}_{\mu,s}$ and $\mu' = \mathbf{q}_{\mu,s}$.

Definition 4 (Probabilistic Forward Simulation). Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times \mathcal{D}(Q')$ such that $(q_*, \delta_{q'_*}) \in \mathcal{S}$ and, for all $(q, \mu') \in \mathcal{S}$ and $(q, l, \mu) \in T$, there exists $\mu' \xRightarrow{l} \mu''$ such that $(\mu, \mu'') \in \mathcal{D}(\mathcal{S})$.

Theorem 1 (from [Seg95]). *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ if and only if there exists a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. The proof is non-trivial and is detailed in Segala's PhD thesis. $\square$

In our setting, the most interesting implication of Theorem 1 is soundness. We will see that the definition of $\sqsubseteq^t$ has some limitations, notably with respect to liveness. To address this, we introduce a stronger notion of trace distribution and strengthen probabilistic forward simulation accordingly. However, the soundness proof of Segala [Seg95] works at the level of $\mathfrak{e}_{\mu,s}$, not s . This is problematic because our strengthening consists in constraining schedulers; thus, we need to adapt the proof. If $\mathcal{S}$ is a probabilistic forward simulation then it induces a choice function $\mathcal{C} : \mathcal{S} \times T \rightarrow Sch \times (\mathcal{S} \rightarrow [0, 1])$ which maps any $(q, \mu') \in \mathcal{S}$ and $(q, l, \mu) \in T$ to the scheduler that induces $\mu' \xRightarrow{l} \mu''$ and the witness w of $(\mu, \mu'') \in \mathcal{D}(\mathcal{S})$.

Algorithm 1 Simulation Algorithm for Schedulers

Input: $\mathcal{P}, \mathcal{P}'$ two PLTSs, $\mathcal{S}$ a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ and $s \in Sch$

```

 $e, e', \mu' \leftarrow q_*, q'_*, \delta_{q'_*}$ 
 $acc \leftarrow e'$ 
 $t \leftarrow s(e)$ 
while  $t \neq \perp$  do
   $s_t, w_t \leftarrow \mathcal{C}(last(e), \mu', t)$ 
   $t' \leftarrow s_t(e')$ 
  while  $t' \neq \perp$  do
     $e' \leftarrow e'.lbl(t').out(t')$ 
     $acc \leftarrow acc.lbl(t').out(t')$ 
     $t' \leftarrow s_t(e')$ 
  end while
   $e', \mu' \leftarrow last(e'), q_{\mu', s_t}$ 
   $e \leftarrow e.lbl(t).q$  with probability  $\frac{\sum_{\mu \in \mathcal{D}(Q')} w(q, \mu) \cdot \mu(e')}{\mu'(e')}$ 
   $t \leftarrow s(e)$ 
end while

```

Once the choice function $\mathcal{C}$ is fixed, Algorithm 1 is a sequential probabilistic program; thus, it can be modelled as a deterministic PLTS. Of course, the validity of the algorithm's individual instructions should be established rigorously, but this is not our concern here. The variable e encodes an execution of $\mathcal{P}$ and acc an execution of $\mathcal{P}'$ that simulates it. In order to extract a scheduler of $\mathcal{P}'$ from Algorithm 1, we need to identify the instructions deciding which transitions to execute in $\mathcal{P}'$. The most obvious one is $t' \leftarrow s_t(e')$ when $t' \neq \perp$; then there is $t \leftarrow s(e)$ when $t = \perp$. The case where $t' = \perp$ has to be discarded because another t' will then be picked without updating acc . When $t = \perp$, the algorithm stops, so the scheduler on $\mathcal{P}'$ also returns $\perp$. After each update of acc , one of the three following events occurs: (i) t is set to $\perp$; (ii) t' is set to a non- $\perp$ value; or (iii) t' is set to $\perp$ indefinitely. Cases (i) and (iii) correspond to scheduling $\perp$ and case (ii) corresponds to scheduling t' . Thus, the key values to compute are $reach(e')$, the probability of reaching a configuration where $acc = e'$, and $sch(e', t)$, the probability of reaching a configuration where $acc = e$ and $t' = t$. Knowing these, one can define a scheduler on $\mathcal{P}'$ as

$$s' : e' \mapsto \begin{cases} t' \mapsto \frac{sch(e', t')}{reach(e')} \\ \perp \mapsto 1 - \frac{\sum_{t' \in T'} sch(e', t')}{reach(e')} \end{cases}$$

We conjecture that $t_{q_*,s} = t_{q'_*,s'}$.

1.2 Non-probabilistic Systems

Non-probabilistic systems can be considered as special instances of PLTSs in which all transitions lead to a Dirac distribution. By identifying each Dirac distribution with the unique state in its support, we may assume that non-probabilistic systems are modelled by LTSs, which leads to several simplifications. In particular, probabilistic forward simulation is equivalent to a simpler simulation relation. On an LTS, we write $q \xRightarrow{l} q'$ if there exists a partial execution from q to q' whose trace is l .

Definition 5 (Weak Simulation). Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times Q'$ such that $(q_*, q'_*) \in \mathcal{S}$ and, for all $(q, q') \in \mathcal{S}$ and $(q, l, p) \in T$, there exists $q' \xRightarrow{l} p'$ such that $(p, p') \in \mathcal{S}$.

Theorem 2. *Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ if and only if there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. According to Theorem 1, it suffices to prove that there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ if and only if there exists a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$. For one direction, it suffices to note that any weak simulation is a probabilistic forward simulation. For the other direction, if $\mathcal{S}$ is a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ then $\{(q, q') \mid \exists (q, \mu) \in \mathcal{S}, q' \in \bar{\mu}\}$ is a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$. $\square$

There is a natural injection of LTSs into PLTSs, but also a natural projection of PLTSs onto LTSs. It consists in transforming all probabilistic non-determinism into classical non-determinism. The projection of a PLTS $\mathcal{P}$, denoted $\rho(\mathcal{P})$, is the LTS whose transitions are defined by $\{(q, l, q') \mid \exists \mu \in \mathcal{D}(Q), (q, l, \mu) \in T \wedge q' \in \bar{\mu}\}$. ρ is trivially the identity on LTSs, and ρ preserves the set of executions and hence the set of traces. As a result, the notion of weak simulation can be lifted to PLTSs: $\mathcal{S}$ is a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ if and only if it is a weak simulation of $\rho(\mathcal{P})$ by $\rho(\mathcal{P}')$. Since ρ is a projection onto LTSs, this is consistent with the previous definition.

Theorem 3. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{T} \subseteq \mathcal{T}'$ only if there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. The proof is folklore. For each execution of $\mathcal{P}$, an execution of $\mathcal{P}'$ can be built inductively so that they share the same trace. $\square$

Probabilistic forward simulation also preserves the set of traces because it induces a forward-backward simulation [LV95]. Forward-backward simulation coincides with trace inclusion.

Definition 6 (Forward-Backward Simulation). Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a forward-backward simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times 2^{Q'}$ such that $(q_*, \{q'_*\}) \in \mathcal{S}$ and, for all $(q, P) \in \mathcal{S}$ and $(q, l, q') \in T$, there exists $P' \subseteq Q'$ such that $(q', P') \in \mathcal{S}$ and, for all $p' \in P'$, there exists $p \in P$ such that $p \xRightarrow{l} p'$.

Theorem 4. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{T} \subseteq \mathcal{T}'$ if and only if there exists a forward-backward simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. See [LV95]. $\square$

Theorem 5. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{T} \subseteq \mathcal{T}'$ only if there exists a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. Let $\mathcal{S}$ be a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$. Define $\mathcal{S}' := \{(q, P) \in Q \times 2^{Q'} \mid \exists \mu \in \mathcal{D}(Q'), (q, \mu) \in \mathcal{S} \wedge \bar{\mu} = P\}$. Then $\mathcal{S}'$ is a forward-backward simulation, so $\mathcal{T} \subseteq \mathcal{T}'$. $\square$

1.3 Preserving Liveness

Our current definitions are not suited to reasoning about liveness. Indeed, $\mathcal{E}$ always contains the empty execution q_* ; thus, $\mathcal{T}$ always contains the empty trace τ . As a result, no liveness property can be satisfied against such a set of executions. However, intuition tells us that such executions should be discarded when evaluating properties, because we implicitly make a progress assumption: “if a transition is enabled then a transition should be performed”.

To take this observation into account, we say that an execution is maximal if it is infinite or its last state is a deadlock. Similarly, we say that a trace is maximal if it is the trace of a maximal execution. According to the progress assumption, properties should be evaluated against the set of maximal traces. Therefore, we must develop simulation relations that preserve the set of maximal traces. The set of maximal traces is denoted $\mathcal{T}^{\max}$.

However, weak simulation does not preserve the set of maximal traces. First, there exists a trivial weak simulation of a deadlock by any LTS. Second, since τ -transitions can be elided, an infinite execution whose trace is τ , called a divergence, can be simulated by stuttering. We call a weak divergence any execution whose trace is τ and which is infinite or ends in a deadlock. We also say that a state weakly diverges if a weak divergence starts from it.

Definition 7 (Maximal Weak Simulation). Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a weak simulation $\mathcal{S}$ of $\mathcal{P}$ by $\mathcal{P}'$ is said to be maximal if, for all $(q, q') \in \mathcal{S}$, whenever q weakly diverges, so does q' .

Theorem 6. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{T}^{\max} \subseteq \mathcal{T}'^{\max}$ only if there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. It suffices to adapt the proof of Theorem 3. When building an execution e' of $\mathcal{P}'$ that simulates a maximal execution e of $\mathcal{P}$, it suffices to check that the additional requirement on the simulation ensures that e' is maximal. If e contains infinitely many non- τ labels then the construction naturally yields an infinite execution. Otherwise, after simulating the smallest finite prefix that contains all non- τ labels, it suffices to trigger the preservation of weak divergence. $\square$

However, this solution is not satisfactory, because the additional requirement asks us to test for the existence of weak divergences in $\mathcal{P}$. In practice, $\mathcal{P}$ will be a PLTS encoding a real implementation whose size and complexity make this search difficult. To circumvent this issue, simulations should rely on tests that can be performed locally (e.g. checking whether a transition is enabled), not globally (e.g. checking whether a divergence exists). We allow global tests on $\mathcal{P}'$ because it will encode a minimal PLTS satisfying a set of properties. We say that a transition is elided by a weak simulation if it is simulated by a stutter.

Definition 8 (Progressive Weak Simulation (inspired by [DSW22])). Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a weak simulation $\mathcal{S}$ of $\mathcal{P}$ by $\mathcal{P}'$ is said to be progressive if

- for all $(q, q') \in \mathcal{S}$, if q is a deadlock then q' weakly diverges;
- there exists $\leq \subseteq Q \times Q$ well-founded such that, for all $(q, q') \in \mathcal{S}$ and $(q, \tau, p) \in T$, if (q, τ, p) is elided by $\mathcal{S}$ then $q > p$ or q' weakly diverges.

Theorem 7. *A weak simulation $\mathcal{S}$ is maximal if and only if it is progressive.*

Proof. Suppose that $\mathcal{S}$ is maximal. Let $(q, q') \in \mathcal{S}$ such that q is a deadlock; then q weakly diverges, so, by maximality, q' weakly diverges. Define $q > p$ if there exists $q' \in Q'$ such that $(q, q') \in \mathcal{S}$, $q \xrightarrow{\tau} p$, $\mathcal{S}$ elides this transition, and q' does not weakly diverge. It remains to check that $>$ is well-founded. If there exists $q_0 > q_1 > \dots$ then q_0 diverges and there exists $q' \in Q'$ such that $(q_0, q') \in \mathcal{S}$ and q' does not weakly diverge, which contradicts the maximality of $\mathcal{S}$.

Suppose that $\mathcal{S}$ is progressive. Let $(q, q') \in \mathcal{S}$ such that q weakly diverges. If $q \xRightarrow{\tau} p$ and p is a deadlock then $q' \xRightarrow{\tau} p'$ with $(p, p') \in \mathcal{S}$; thus, p' weakly diverges and so does q' . If q diverges then either q' weakly diverges, or there exists $q' \xRightarrow{\tau} p'$ such that p' simulates a suffix of q 's divergence by stuttering, contradicting the well-foundedness of $>$. $\square$

Unfortunately, progressive weak simulations are not preserved by composition: if there is a progressive weak simulation of $\mathcal{P}$ by $\mathcal{P}'$, there might not be a progressive weak simulation of $\mathcal{P} \parallel_S \mathcal{P}''$ by $\mathcal{P}' \parallel_S \mathcal{P}''$. Parallel composition might create new deadlocks because of synchronisation. To avoid these issues, we use the IO-automata framework of [LT26].

Definition 9 (Input/Output Automaton). An Input/Output automaton is a PLTS whose set of labels is partitioned into two subsets: input labels In and output labels Out , such that, for all states $q \in Q$ and $in \in In$, there is $\mu \in \mathcal{D}(Q)$ such that $(q, in, \mu) \in T$.

Theorem 8. Let $\mathcal{P}$, $\mathcal{P}'$ and $\mathcal{P}''$ be IO-automata such that $Out \cap Out'' = \emptyset$. If there exists a progressive weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ then there exists a progressive weak simulation of $\mathcal{P} \parallel_S \mathcal{P}''$ by $\mathcal{P}' \parallel_S \mathcal{P}''$.

Proof. Let us denote by $\mathcal{S}$ the progressive weak simulation of $\mathcal{P}$ by $\mathcal{P}'$. Then $\mathcal{S}' := \{((q, q''), (q', q'')) \mid (q, q') \in \mathcal{S}\}$ is a weak simulation of $\mathcal{P} \parallel_S \mathcal{P}''$ by $\mathcal{P}' \parallel_S \mathcal{P}''$. To prove that $\mathcal{S}'$ is progressive, it suffices to note that if there is a weak divergence in $\mathcal{P} \parallel_S \mathcal{P}''$ from (q, q'') then q or q'' weakly diverges. This holds because of input-enabledness and the compatibility of IO-automata. It then suffices to apply the maximality of $\mathcal{S}$. $\square$

Since our main simulation relation is probabilistic forward simulation, we need to adapt progressiveness to it. We have seen that probabilistic forward simulation preserves the set of traces because it induces a forward-backward simulation. Thus, we can start by adapting progressiveness to forward-backward simulations. A forward-backward simulation is said to be maximal if, for all $(q, P) \in \mathcal{S}$, whenever q weakly diverges, there exists $p \in P$ that weakly diverges.

Theorem 9. Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{T}^{\max} \subseteq \mathcal{T}'^{\max}$ if and only if there exists a maximal forward-backward simulation of $\mathcal{P}$ by $\mathcal{P}'$.

Proof. Based on the construction in the proof of Theorem 4. $\square$

For the same reason as for weak simulation, testing for weak divergences in $\mathcal{P}$ should be avoided.

Definition 10 (Progressive Forward-Backward Simulation). Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, a forward-backward simulation $\mathcal{S}$ of $\mathcal{P}$ by $\mathcal{P}'$ is said to be progressive if

- for all $(q, P) \in \mathcal{S}$, if q is a deadlock then there exists $p \in P$ that weakly diverges;
- there exists $< \subseteq Q \times Q$ well-founded such that, for all $(q, P) \in \mathcal{S}$ and $(q, \tau, q') \in T$, $q > q'$ or there exists $p \in P$ that weakly diverges or there exists $P' \subseteq Q'$ such that $(q', P') \in \mathcal{S}$ and, for all $p' \in P'$, there exists $p \in P$ such that $p \xRightarrow{\tau} p'$ which is not an elision.

Theorem 10. *A forward-backward simulation $\mathcal{S}$ is maximal if and only if it is progressive.*

Proof. Same proof as for Theorem 7. $\square$

Now, it suffices to modify the definition of probabilistic forward simulation so that the construction of Theorem 5 produces a progressive forward-backward simulation.

Definition 11 (Progressive Probabilistic Forward Simulation). Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, a progressive probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times \mathcal{D}(Q')$ such that $(q_*, \delta_{q_*}) \in \mathcal{S}$ and, for all $(q, \mu') \in \mathcal{S}$ and $(q, l, \mu) \in T$, there exists $\mu' \xRightarrow{l} \mu''$ such that $(\mu, \mu'') \in \mathcal{D}(\mathcal{S})$. Moreover, if q is a deadlock then there exists $q' \in \bar{\mu}'$ that weakly diverges. Furthermore, there exists $>\subseteq Q \times Q$ well-founded such that, for all $(q, \mu') \in \mathcal{S}$ and $(q, \tau, \mu) \in T$ and $p \in \bar{\mu}$, $q > p$ or there exists $q' \in \bar{\mu}'$ that weakly diverges or there exists $\mu_p \in \mathcal{D}(Q)$ such that $w(p, \mu_p) > 0$ where w is the witness of $(\mu, \mu'') \in \mathcal{S}$ and, for all $q' \in \bar{\mu}' \cap \bar{\mu}_p$, $s(q')(\perp) = 0$ where s is the scheduler s that induces $\mu' \xRightarrow{\tau} \mu''$.

Unfortunately, this is not enough to preserve liveness at the level of schedulers. Indeed, the current definition does not preserve maximality of schedulers because it is enough that one state is the probability distribution weakly diverges. To solve that issue, it suffices to replace "there exists $q' \in \bar{\mu}'$ that weakly diverges" by "for all $q' \in \bar{\mu}'$, q' weakly diverges".

1.4 Partial Information

We encode partial information by restrictions on the set of schedulers. Intuitively, if the scheduler has not access to all information then it might not be able to distinguish some states or some labels, thus, some distinct executions might look the same to it and it should schedule the same probability distribution of transitions.

Definition 12 (Adversary). An adversary is a couple of observation function $(\mathcal{O}_Q, \mathcal{O}_L)$ which respectively map states and labels to state observation and label observation such that, for all $t, t' \in T$, if $\text{src}(t) = \text{src}(t')$ and $\mathcal{O}_L(\text{lbl}(t)) = \mathcal{O}_L(\text{lbl}(t'))$ then $t = t'$.

The injectivity of $\mathcal{O}_L$ on the set of transitions enabled in a given state ensures that the partial-information does not prevent the adversary from resolving non-determinism. An adversary $(\mathcal{O}_Q, \mathcal{O}_L)$ induces an observation $\mathcal{O}_{\mathcal{E}}$ on executions defined by $\mathcal{O}_{\mathcal{E}}(q_0.l_0 \dots) = \mathcal{O}_Q(q_0).\mathcal{O}_L(l_0) \dots$. An scheduler s is consistent with an adversary if, for all $e, e' \in \mathcal{E}^*$, $\mathcal{O}_{\mathcal{E}}(e) = \mathcal{O}_{\mathcal{E}}(e') \implies s(e) = s(e')$. The omniscient adversary is encoded by setting observation functions to identity.

Our simulations are sound for partial-information adversaries because they were designed to be sound for omniscient adversaries. However, there is a completeness gap as partial-information makes more reduction sound. Our chosen solution is to build a PLTS on which omniscient adversary is equivalent to partial-information adversary on the original PLTS. This is the belief construction.

Definition 13 (Belief PLTS). Given a PLTS $\mathcal{P}$ and an adversary $\mathcal{A} := (\mathcal{O}_Q, \mathcal{O}_L)$, the belief PLTS $\mathcal{P}_{\mathcal{A}}$ is defined as the tuple $(\{\mu \in \mathcal{D}(Q) \mid \mathcal{O}_Q(\bar{\mu}) = \{\cdot\}\}, \delta_{q_*}, \mathcal{O}_L(L), T_{\mathcal{A}})$ where

$$\mu \xrightarrow{l} \mu' \in T_{\mathcal{A}} \iff \exists (t_q)_{q \in \bar{\mu}} \in T^{\bar{\mu}}, \begin{cases} \forall q \in \bar{\mu}, \text{src}(t_q) = q \wedge \mathcal{O}_L(\text{lbl}(t_q)) = l \\ \mu' = (\mu_o := q' \mapsto \frac{\mathbb{1}_{\mathcal{O}_Q(q')=o} \cdot \sum_{q \in \bar{\mu}} \mu_q(q')}{\sum_{q, q' \in Q} \mathbb{1}_{\mathcal{O}_Q(q')=o} \cdot \mu_q(q')}) \mapsto \sum_{q, q' \in Q} \mathbb{1}_{\mathcal{O}_Q(q')=o} \cdot \mu_q(q') \end{cases}$$

Theorem 11. *Given a PLTS $\mathcal{P}$ and an adversary $\mathcal{A}$, the set of trace distribution of $\mathcal{P}$ induced by a scheduler consistent with $\mathcal{A}$ is equal to $\text{tdist}(\mathcal{P})$.*

2 List of the Protocols

Common Notation. Throughout this section, Id denotes the set of process identifiers with $n := |Id|$, $F \subseteq Id$ denotes the (a priori unknown) set of faulty processes with $f \geq |F|$ a fixed upper bound, and $Id \setminus F$ is the set of correct processes. For each protocol we provide two implementations:

- an *ideal implementation* (**Spec**), designed for a no-delay system with crash failures and mediated by a trusted third party;
- a *concrete implementation* (**Impl**), designed for an asynchronous system with Byzantine failures.

Messages exchanged with the third party (resp. between processes) are written $\langle \text{TAG}, \cdot \rangle$, state variables stay in plain math, and arrays indexed by process identifiers are written $v[id]$.

2.1 Main Protocol: Asynchronous Byzantine Agreement (ABA)

(taken from [ABDY22]) Processes start with a bit $\{0, 1\}$ and may return a bit. A program implements ABA if it satisfies three properties: Validity, Agreement, Almost-Sure Termination.

- Validity: if a correct process returns b then a correct process had $1 - b$ as input.
- Agreement: if two correct processes return, then their outputs are equal.
- Termination: if $n - f$ correct processes take part in the protocol then, with probability 1, all correct processes will eventually return.

Protocol 1. Ideal implementation. The trusted third party collects each process's input bit and, once at least $n - f$ contributions support some bit b , fixes the bound value to b and broadcasts $\langle \text{OUTPUT}, b \rangle$ to all processes. Late callers are coerced to the bound value or to a random draw from a non-Dirac distribution μ , which preserves the bound while modelling the adversary's ability to skew honest participation.

Algorithm 2 Ideal Implementation of ABA

Input: $b \in \{0, 1\}$

Code for every process:

send $\langle \text{INPUT}, b \rangle$ to the third party

upon receiving $\langle \text{OUTPUT}, b \rangle$ from the third party:

return b

Code for the third party:

$bind, call \leftarrow \perp, (\perp)^{Id}$

upon receiving $\langle \text{INPUT}, b \rangle$ from a process id and $bind = \perp$:

$call[id] \leftarrow b$

upon $|F \cup \{id \in Id \setminus F \mid call[id] = b\}| \geq n - f$:

$bind, call \leftarrow b, (\perp)^{Id}$

if $\{id \in Id \setminus F \mid call[id] = 1 - b\} = \emptyset$:

send $\langle \text{OUTPUT}, b \rangle$ to all

upon $call[id] = \perp$ and $bind \neq \perp$:

$call[id] \leftarrow bind$ **or** μ

$\triangleright \mu$ is any non-Dirac distribution over $\{0, 1\}$

Protocol 2. *Concrete implementation.* Round-based protocol combining Graded Binding Crusader Agreement (GBCA) with a Weak Common Coin (WCC). Each round, the process refines its bit through GBCA and tosses a coin via WCC; it adopts the coin if GBCA was undecided ($b = \perp$) and otherwise decides on b when GBCA returned the highest grade A , while continuing to run to help the others terminate.

Algorithm 3 Implementation of ABA taken from [ABDY22]

Input: $b \in \{0, 1\}$

$r \leftarrow 1$

$decided \leftarrow \perp$

loop

$(b, g) \leftarrow \text{GBCA}_r(b)$ $\triangleright b \in \{0, 1, \perp\}, g \in \{A, B, C\}$

$c \leftarrow \text{WCC}_r()$ $\triangleright$ common coin for round r

if $b = \perp$ **then**

$b \leftarrow c$

else if $g = A$ **then**

$decided \leftarrow b$ $\triangleright$ keep participating to help others

end if

$r \leftarrow r + 1$

end loop

$\triangleright$ TODO: verify termination rule

Definition 14. $\text{ABA.Concrete} := \text{ABA.Impl}[\text{GBCA.Impl}, \text{WCC.Impl}]$, the instantiation of ABA.Impl with concrete implementations of the sub-protocols.

Definition 15. $\text{ABA.Hybrid} := \text{ABA.Impl}[\text{GBCA.Spec}, \text{WCC.Spec}]$, the instantiation of ABA.Impl with ideal specifications of the sub-protocols.

2.2 ABA Sub-Protocols

Graded Binding Crusader Agreement (GBCA) (taken from [ABDY22, AW23]) Processes start with a binary input $\{0, 1\}$ and may return a value in $\{(0, A), (0, B), (\perp, C), (1, B), (1, A)\}$. A program implements GBCA if it satisfies four properties: Validity, Graded Agreement, Binding, Termination.

- Validity: if a correct process does not return (b, A) then a correct process had $1 - b$ as input.
- Graded Agreement: if two correct processes return (b, X) and (b', X') then $b = 0 \implies b' \neq 1$ and $X = A \implies X' \neq \perp$.
- Binding: when a correct process returns then a bound value b is defined and, in all extensions of the execution, correct processes do not return $(1 - b, B)$ or $(1 - b, A)$.
- Termination: if $n - f$ correct processes take part in the protocol then all correct processes will eventually return.

Protocol 3. *Ideal implementation.* The trusted third party tallies the contributions and, as soon as a quorum supports some bit b , fixes $bind$ to b and sets the global grade: A if no correct process input $1 - b$, otherwise C . Each process is then served either the highest grade pair $(bind, A)$ (resp. $(\perp, C)$) or the conservative $(bind, B)$ as chosen non-deterministically by the adversary.

Algorithm 4 Ideal Implementation of GBCA

Input: $b \in \{0, 1\}$

Code for every process:

send $\langle \text{INPUT}, b \rangle$ to the third party

upon receiving $\langle \text{OUTPUT}, (b, g) \rangle$ from the third party:

return (b, g)

Code for the third party:

$bind, grade, call \leftarrow \perp, \perp, (\perp)^{Id}$

upon receiving $\langle \text{INPUT}, b \rangle$ from a process id :

$call[id] \leftarrow b$

upon $|F \cup \{id \in Id \setminus F \mid call[id] = b\}| \geq n - f$:

$bind \leftarrow b$

if $\{id \in Id \setminus F \mid call[id] = 1 - b\} = \emptyset$:

$grade \leftarrow A$

else

$grade \leftarrow C$

upon $bind \neq \perp$ and $grade = A$ and not sent $\langle \text{OUTPUT}, \cdot \rangle$ to process id :

send $\langle \text{OUTPUT}, (bind, B) \rangle$ or $\langle \text{OUTPUT}, (bind, A) \rangle$ to process id

upon $bind \neq \perp$ and $grade = C$ and not sent $\langle \text{OUTPUT}, \cdot \rangle$ to process id :

send $\langle \text{OUTPUT}, (bind, B) \rangle$ or $\langle \text{OUTPUT}, (\perp, C) \rangle$ to process id

Protocol 4. *Concrete implementation.* Requires no sub-protocol. Parameters: n processes with at most $f < n/3$ corrupted, each holding an input bit $b \in \{0, 1\}$. After an input-amplification phase, the protocol runs three Bracha-style rounds of $\langle \text{ECHO}, \cdot \rangle$, $\langle \text{VOTE}, \cdot \rangle$, $\langle \text{BIND}, \cdot \rangle$ messages to settle the bound value and assign the grade.

Weak Common Coin (WCC) Processes start with no input and may return a bit. A program implements WCC if it satisfies three properties: ϵ -Goodness, Unpredictability, Termination.

- ϵ -Goodness: with probability ϵ , all correct processes return 0 (resp. 1).
- Unpredictability: until $f + 1$ processes have called WCC, all outcomes are still possible.
- Termination: if $n - f$ correct processes take part in the protocol then all correct processes will eventually return.

Protocol 5. *Ideal implementation.* The trusted third party samples a coin r from a distribution μ such that $\mu(0) = \mu(1) = \epsilon$ and $\mu(\perp) = 1 - 2\epsilon$. If $r \in \{0, 1\}$ the same value is broadcast to every process; otherwise each process is delivered an adversarially-chosen bit, modelling the worst case where the coin does not coordinate the honest parties.

Protocol 6. *Concrete implementation.* Requires Gather and RSD. Details to be determined.

Definition 16. $\text{WCC.Concrete} := \text{WCC.Impl}[\text{Gather.Concrete}, \text{RSD.Concrete}]$, the instantiation of WCC.Impl with fully concrete sub-protocols.

Definition 17. $\text{WCC.Hybrid} := \text{WCC.Impl}[\text{Gather.Spec}, \text{RSD.Spec}]$, the instantiation of WCC.Impl with ideal specifications of the sub-protocols.

Algorithm 5 Implementation of GBCA taken from [ABDY22]

Input: $b \in \{0, 1\}$

send $\langle \text{INPUT}, b \rangle$ to all processes

upon receiving $\langle \text{INPUT}, b \rangle$ from at least $f + 1$ processes and not sent $\langle \text{INPUT}, b \rangle$ yet:
send $\langle \text{INPUT}, b \rangle$ to all

upon receiving $\langle \text{INPUT}, b \rangle$ from at least $n - f$ processes:
 $Approved \leftarrow Approved \cup \{b\}$
if not sent $\langle \text{ECHO}, \cdot \rangle$ yet **then** send $\langle \text{ECHO}, b \rangle$ to all

wait until
(a) receiving $\langle \text{ECHO}, b \rangle$ from at least $n - f$ processes **or**
(b) receiving $\langle \text{ECHO}, \cdot \rangle$ from at least $n - f$ processes and $|Approved| > 1$
if (a) **then** send $\langle \text{VOTE}, b \rangle$ to all **else** send $\langle \text{VOTE}, \perp \rangle$ to all

wait until
(a) receiving $\langle \text{VOTE}, b \rangle$ from at least $n - f$ processes **or**
(b) receiving $\langle \text{VOTE}, \cdot \rangle$ from at least $n - f$ processes and $|Approved| > 1$
if (a) **then** send $\langle \text{BIND}, b \rangle$ to all **else** send $\langle \text{BIND}, \perp \rangle$ to all

wait until
(a) receiving $\langle \text{BIND}, b \rangle$ from at least $n - f$ processes **or**
(b) receiving $\langle \text{BIND}, \cdot \rangle$ from at least $n - f$ processes and there exists $b \in \{0, 1\}$ such that $\langle \text{BIND}, b \rangle$ has been received once and $\langle \text{VOTE}, b \rangle$ has been received from $f + 1$ processes and $|Approved| > 1$ **or**
(c) receiving $\langle \text{BIND}, \perp \rangle$ from at least $n - f$ processes and $|Approved| > 1$
if (a) **then** return (b, A) **elif** (b) **then** return (b, B) **else** return $(\perp, C)$

Algorithm 6 Ideal Implementation of WCC

Input: none

Code for every process:
send $\langle \text{INPUT} \rangle$ to the third party
upon receiving $\langle \text{OUTPUT}, b \rangle$ from the third party:
return b

Code for the third party:
 $r \leftarrow \mu$ $\triangleright \mu: 1 \mapsto \epsilon, 0 \mapsto \epsilon, \perp \mapsto 1 - 2\epsilon$
if $r \neq \perp$:
send $\langle \text{OUTPUT}, r \rangle$ to all
else
for all $id \in Id$, send $\langle \text{OUTPUT}, 0 \rangle$ **or** $\langle \text{OUTPUT}, 1 \rangle$ to process id

Algorithm 7 Implementation of WCC (simplified from [FKT22])

Input:

Sub-Protocols: n instances of $SRSD$ (process id is the subject of instance $SRSD_{id}$), 1 instance of Gather

$\forall id \in Id, SRSD_{id}.Asg()$

wait until $ValueAssigned_{id}$

Gather.call(id)

upon Gather.return(S)

$\forall id \in S, SRSD_{id}.Ret.call()$

$T \leftarrow \emptyset$

upon $SRSD_{id}.Ret.return(x)$ and $id \in S$

$T \leftarrow T \cup \{x\}$

upon $|T| = |S|$

return 0 if $0 \in T$ else 1

2.3 WCC Sub-Protocols

Gather (taken from [SA21, SA24, AFW25]) Processes start with an input $x \in X$ and may return a partial function $f : Proc \rightarrow X$. A program implements Gather if it satisfies four properties: Validity, Agreement, Common Core, Binding, Termination.

- Validity: if a correct process returns f and $f(q)$ is defined and q is correct then $f(q)$ is q 's input.
- Agreement: if two correct processes return f and g then $f(q)$ and $g(q)$ are equal or one of them is undefined.
- Binding Common Core: once a correct process has returned, there exists $S \subseteq Proc$ of size at least $n - f$ such that all f returned by a correct process in the future is defined on S .
- Termination: if $n - f$ correct processes take part in Gather then all correct processes will eventually return.

Protocol 7. Ideal implementation. The trusted third party records each process's input, and once a quorum of correct contributions has been received, non-deterministically commits to a binding common core $bind \subseteq Id$. Each process is then served a partial function whose domain S is non-deterministically extended above $bind$, returning the recorded inputs of correct members of S and arbitrary values for corrupted members.

Protocol 8. Concrete implementation. Requires $|Id|$ instances of BRB (process id is leader in BRB_{id}). Each process first broadcasts its input via its own BRB instance, then amplifies the set of approved pairs AP through three rounds of $\langle ECHO, \cdot \rangle$, $\langle VOTE, \cdot \rangle$, $\langle BIND, \cdot \rangle$ messages before returning the union of the approved sets.

Definition 18. $Gather.Concrete := Gather.Impl[BRB.Impl]$, the instantiation of $Gather.Impl$ with the concrete implementation of BRB.

Definition 19. $Gather.Hybrid := Gather.Impl[BRB.Spec]$, the instantiation of $Gather.Impl$ with the ideal specification of BRB.

Algorithm 8 Ideal Implementation of Gather

Input: m

Code for every process:

send $\langle \text{INPUT}, m \rangle$ to the third party

upon receiving $\langle \text{OUTPUT}, f \rangle$ from the third party:

return f

Code for the third party:

$call, bind \leftarrow \emptyset, \perp$

upon receiving $\langle \text{INPUT}, m \rangle$ from process id :

$call \leftarrow call \cup \{(id, m)\}$

upon $|F \cup \{id \in Id \setminus F \mid (id, \cdot) \in call\}| \geq n - f$ and $bind = \perp$:

$bind \leftarrow S$ for some $S \subseteq F \cup \{j \in Id \setminus F \mid (j, \cdot) \in call\}$ $\triangleright$ Non-deterministic choice

upon $bind \neq \perp$ and not sent $\langle \text{OUTPUT}, \cdot \rangle$ to process id :

select $bind \subseteq S \subseteq F \cup \{j \in Id \setminus F \mid (j, \cdot) \in call\}$ and

send $\langle \text{OUTPUT}, \{(j, m) \in call \mid j \in S \setminus F\} \cup \{(j, \cdot) \mid j \in S \cap F\} \rangle$ to process id

Algorithm 9 Implementation of Gather from [AFW25]

Input: m

Sub-Protocols: $|Id|$ instances of BRB (process id is leader in BRB_{id})

$BRB_{id}.call(m)$

upon $BRB_i.return(m')$:

$AP \leftarrow AP \cup \{(i, m')\}$

upon $|AP| \geq n - f$ and not sent $\langle \text{ECHO}, \cdot \rangle$:

send $\langle \text{ECHO}, AP \rangle$ to all

upon receiving approved $\langle \text{ECHO}, AP_{id} \rangle$ from at least $n - f$ processes and not sent $\langle \text{VOTE}, \cdot \rangle$:

send $\langle \text{VOTE}, \bigcup_{id} AP_{id} \rangle$ to all

upon receiving approved $\langle \text{VOTE}, AP_{id} \rangle$ from at least $n - f$ processes and not sent $\langle \text{BIND}, \cdot \rangle$:

send $\langle \text{BIND}, \bigcup_{id} AP_{id} \rangle$ to all

upon receiving approved $\langle \text{BIND}, AP_{id} \rangle$ from at least $n - f$ processes:

return $\bigcup_{id} AP_{id}$

Random Secret Draw (RSD) The RSD protocol is composed of two operations: Assignment and Retrieval. Assignment assigns a random, secret value to each process and Retrieval reveals these values to the processes. Moreover, the API of RSD provides a notification, for each process, recording which processes have a value assigned.

- Assignment Validity: if all correct processes call Assignment, then all correct processes will eventually receive a notification for every correct process.
- Assignment Totality: if a correct process receives a notification for process id then all correct processes will get such a notification.
- Randomness: if no correct process has called Retrieval then the value returned by Retrieval for any process follows the uniform distribution.
- Secrecy: if no correct process has called Retrieval then the adversary has no information about the return value of Retrieval.
- Retrieval Termination: if all correct processes call Retrieval then the operation terminates for all correct processes.

Protocol 9. Ideal implementation. The trusted third party tracks which processes have called Assignment; once a quorum is reached, it issues an $\langle \text{ASSIGN}, id \rangle$ notification for every $id \in Id$. On Retrieval, after enough processes have asked, it samples a fresh value μ uniformly over the output domain and delivers it to every process.

Algorithm 10 Ideal Implementation of RSD

Operation: $RSD.Asg()$

Code for every process:

send $\langle \text{INPUT} \rangle$ to the third party.

upon receiving $\langle \text{ASSIGN}, id \rangle$ from the third party:

event $ValueAssigned(id)$

Code for the third party:

$call \leftarrow (\perp)^{Id}$

upon receiving $\langle \text{INPUT} \rangle$ from process id :

$call[id] \leftarrow \top$

upon $|F \cup \{id \in Id \setminus F \mid call[id] = \top\}| \geq n - f$:

$\forall id \in Id$, send $\langle \text{ASSIGN}, id \rangle$ to all

Operation: $RSD.Ret()$

Code for every process:

send $\langle \text{RETRIEVE} \rangle$ to the third party.

upon receiving $\langle \text{OUTPUT}, x \rangle$ from the third party:

 return x

Code for the third party:

$call', out \leftarrow (\perp)^{Id}, (\perp)^{Id}$

upon receiving $\langle \text{RETRIEVE} \rangle$ from process id :

$call'[id] \leftarrow \top$

upon $|F \cup \{id \in Id \setminus F \mid call'[id] = \top\}| \geq n - f$ and $out[id] = \perp$:

$out[id] \leftarrow \mu$

 send $\langle \text{OUTPUT}, out[id] \rangle$ to all

Protocol 10. Concrete implementation. Requires $|Id|$ instances of BRB and $|Id|^2$ instances of AVSS. Each process id draws a fresh secret $x_{id'}$ for every $id' \in Id$ and shares it through an AVSS instance, then uses its own BRB to publish the set S of dealers it received shares from. A process is assigned a value once it has collected, for each entry of the committed set S' , the corresponding AVSS sharing return; the retrieved value is the sum of the AVSS reconstructions.

Algorithm 11 Implementation of RSD from [FKT22]

Sub-Protocols: $|Id|$ instances of BRB (process id is leader in BRB_{id}) and $|Id|^2$ instances of AVSS (process id is dealer in $AVSS_{id}^k$ for each $k \in Id$)

Operation: $RSD.Asg()$

$S \leftarrow \emptyset$

for all $id' \in Id$:

$x_{id'} \leftarrow \mu$ $\triangleright \mu$: uniform distribution over the domain of secrets
 $AVSS_{id'}^{id'}.Sh.call(x_{id'})$

upon $AVSS_{id'}^{id'}.Sh.return()$:

$S \leftarrow S \cup \{id'\}$

upon $|S| \geq n - f$ and not called BRB_{id} :

$BRB_{id}.call(S)$

upon $BRB_{id'}.return(S')$ and $\forall k \in S', AVSS_k^{id'}.Sh.return()$:

$src[id'] \leftarrow S'$

event $ValueAssigned(id')$

$\forall k \in S', AVSS_k^{id'}.Rec.call()$

Operation: $RSD.Rec()$

return $id' \mapsto \sum_{k \in src[id']} AVSS_k^{id'}.Rec.return()$

Definition 20. $RSD.Concrete := RSD.Impl[BRB.Impl, AVSS.Spec]$, the instantiation of $RSD.Impl$ with the concrete implementation of BRB. AVSS is used as a black-box, so its specification is plugged in directly.

Definition 21. $RSD.Hybrid := RSD.Impl[BRB.Spec, AVSS.Spec]$, the instantiation of $RSD.Impl$ with the ideal specifications of BRB and AVSS.

2.4 Basic Primitives

Byzantine Reliable Broadcast (BRB) (taken from [Bra87]) Each instance of BRB has a designated *leader* with an input message m and processes may return a received message. A program implements BRB if it satisfies three properties: Validity, Totality and Agreement.

- Validity: if the leader is not corrupted then all correct processes must return its input.
- Agreement: if two correct processes return then they return the same value.
- Totality: if a correct process returns then all correct processes eventually return.

Protocol 11. Ideal implementation. The trusted third party receives the leader's input and broadcasts it to every process when the leader is correct; otherwise it broadcasts an arbitrary message m' chosen non-deterministically.

Algorithm 12 Ideal Implementation of BRB

Input: $m \in \{0, 1\}^*$ for the leader

Code for the leader:

send $\langle \text{INPUT}, m \rangle$ to the third party

Code for every process:

upon receiving $\langle \text{OUTPUT}, m \rangle$ from the third party:
return m

Code for the third party:

upon receiving $\langle \text{INPUT}, m \rangle$ from leader:

if leader is correct:

send $\langle \text{OUTPUT}, m \rangle$ to all

else

send $\langle \text{OUTPUT}, m' \rangle$ to all

$\triangleright$ Arbitrary m'

Protocol 12. *Concrete implementation.* Requires no sub-protocol. Parameters: n processes with at most $f < n/3$ corrupted; leader ℓ with input m . Standard Bracha broadcast: $\langle \text{INIT}, \cdot \rangle$ from the leader is echoed by all processes; an $n - f$ echo quorum or an $f + 1$ ready quorum triggers $\langle \text{READY}, \cdot \rangle$; a $2f + 1$ ready quorum triggers delivery.

Algorithm 13 Implementation of BRB taken from [Bra87]

Code for the leader ℓ with input m :

send $\langle \text{INIT}, m \rangle$ to all processes

Code for every process:

upon first reception of $\langle \text{INIT}, m \rangle$ from ℓ **do**

send $\langle \text{ECHO}, m \rangle$ to all processes

upon reception of $\langle \text{ECHO}, m \rangle$ from at least $n - f$ distinct processes **do**

if $\langle \text{READY}, \cdot \rangle$ has not yet been sent **then**

send $\langle \text{READY}, m \rangle$ to all processes

upon reception of $\langle \text{READY}, m \rangle$ from at least $f + 1$ distinct processes **do**

if $\langle \text{READY}, \cdot \rangle$ has not yet been sent **then**

send $\langle \text{READY}, m \rangle$ to all processes

upon reception of $\langle \text{READY}, m \rangle$ from at least $2f + 1$ distinct processes **do**

return m

Asynchronous Verifiable Secret Sharing (AVSS) AVSS is actually a pair of protocols (Sh, Ret) with a designated *dealer* owning a secret s . A program implements a $(1 - \epsilon)$ -correct AVSS if it satisfies six properties: Validity, Sharing Termination, Sharing Totality, Retrieval Termination, Commitment, Secrecy.

- Validity: if the dealer is correct and a correct process returns from *Ret* then the return value is the dealer's secret.
- Sharing Termination: if the dealer is correct and calls *Sh* then all correct processes will eventually return from *Sh*.

- **Sharing Totality:** if a correct process returns from Sh then all correct processes will eventually return from Sh .
- **Retrieval Termination:** if all correct processes call Ret then all correct processes will eventually return from Ret .
- **Commitment:** if a correct process returns from Sh then there exists a secret s such that correct processes can only retrieve s from that point.
- **Secrecy:** if the dealer is correct and no correct process has called Ret then the adversary has no information about the future retrieved value.

Protocol 13. Ideal implementation. The trusted third party stores the dealer's secret (or an adversarially-chosen one if the dealer is corrupted) and immediately notifies every process that sharing has completed. On Retrieval, as soon as more than f processes have asked, the stored secret is broadcast to all processes.

Algorithm 14 Ideal Implementation of AVSS

Input: $s \in \{0,1\}^*$ for the dealer

Operation: $AVSS.Sh()$

Code for the dealer:

send $\langle SH.INPUT, s \rangle$ to the third party

Code for every process:

upon receiving $\langle SH.OUTPUT \rangle$ from the third party:
return

Code for the third party:

upon receiving $\langle SH.INPUT, s \rangle$ from dealer or dealer is corrupted:

$secret \leftarrow s$ $\triangleright s$ is chosen non-deterministically if no message received

send $\langle SH.OUTPUT \rangle$ to all

Operation: $AVSS.Ret()$

Code for every process:

send $\langle RET.INPUT \rangle$ to the third party

upon receiving $\langle RET.OUTPUT, m \rangle$ from the third party:
return m

Code for the third party:

$call \leftarrow (\perp)^{Id}$

upon receiving $\langle RET.INPUT \rangle$ from process id :

$call[id] \leftarrow \top$

upon $|F \cup \{id \in Id \setminus F \mid call[id] = \top\}| \geq f + 1$:

send $\langle RET.OUTPUT, secret \rangle$ to all

AVSS is used as a black-box, so no concrete implementation is provided.

2.5 Summary of Properties

The linear safety properties are preserved by Segala's probabilistic forward simulation. The branching safety properties can be transformed into linear ones by enhancing the interface with

an external binding label. Segala’s simulation does not preserve liveness by default. However, it can be strengthened following [DSW22] to preserve liveness in a complete manner.

	Linear Safety	Branching Safety	Liveness
ABA	Validity & Agreement	Binding	Termination
GBCA	Validity & Agreement		Termination
WCC	Validity & Agreement	Binding Common Core	Termination
Gather			Assignment Validity & Assignment Totality & Retrieval Termination
RSD			Totality
BRB	Validity & Agreement	Commitment	Sharing Termination & Sharing Totality & Retrieval Termination
AVSS	Validity		

Table 1: Table of CTL Properties

Segala’s simulation was designed to be sound and complete for trace distributions so it preserves probabilistic safety properties. Note that Unpredictability is a probabilistic safety property. For probabilistic liveness, we will rely on the liveness preservation as it is sound (but not complete). The lack of completeness is not an issue because the limitation of the termination proof rule is probabilistic convergence, which is never dealt with by our simulations. The probabilistic convergence argument will only be necessary to prove that the ideal implementation of ABA almost-surely terminates and can be discharged to the super-Martingale proof rule of [EMMS26]. For Secrecy, we use the same trick as for branching: enhancing the interface.

	Probabilistic Safety	Probabilistic Liveness	Secrecy
ABA	ϵ -Goodness & Unpredictability	Almost-Sure Termination	
GBCA			
WCC			
Gather	Randomness		
RSD			Secrecy
BRB			
AVSS			Secrecy

Table 2: Table of non-CTL Properties

3 Proof Decomposition

The end goal is the correctness of `ABA.Impl`. We establish it via a simulation by `ABA.Spec` and transport the spec’s correctness. The simulation itself is decomposed recursively along the sub-protocol structure.

For a protocol P with concrete sub-protocols $Q_1, \dots, Q_k$, the step “ $P.Spec$ simulates $P.Impl$ ” is split into:

1. (Substitution) $P.Impl[Q_1.Spec, \dots, Q_k.Spec]$ is simulated by $P.Impl$, using the child simulations $Q_i.Spec$ simulates $Q_i.Impl$ and compositionality.

2. (Core) $P.Spec$ simulates the hybrid $P.Impl[Q_1.Spec, \dots, Q_k.Spec]$.
3. (Transitivity) Compose (1) and (2).

Leaves of the recursion (no sub-protocol, or only the black-box $AVSS.Spec$) skip the substitution step.

3.1 Top-level

Theorem 12. *ABA.Concrete satisfies Validity, Agreement and Almost-Sure Termination.*

Proof. Soundness of probabilistic forward simulation (Theorem ??) applied to 13 yields $ABA.Concrete \sqsubseteq_{TD} ABA.Spec$, which transports the safety properties (Validity and Agreement) of $ABA.Spec$ to $ABA.Concrete$. Almost-Sure Termination is a probabilistic liveness property and is handled separately via the liveness-preserving strengthening of $\preceq_{PFS}$. $\square$

3.2 ABA

Theorem 13. *ABA.Spec simulates ABA.Concrete.*

Proof. By transitivity of probabilistic forward simulation (Lemma ??), chain 15 with 14 by transitivity. $\square$

Lemma 14. *ABA.Hybrid is simulated by ABA.Concrete.*

Proof. $ABA.Concrete$ and $ABA.Hybrid$ share the same skeleton $ABA.Impl$ and differ only in their sub-protocol instances. Apply the pre-congruence of $\preceq_{PFS}$ for parallel composition (Theorem ??) to the child simulations 19 and 16, lifted along the $ABA.Impl$ context. $\square$

Lemma 15. *ABA.Spec simulates ABA.Hybrid.*

3.3 WCC

Theorem 16. *WCC.Spec simulates WCC.Concrete.*

Proof. By transitivity of probabilistic forward simulation (Lemma ??), chain 18 with 17 by transitivity. $\square$

Lemma 17. *WCC.Hybrid is simulated by WCC.Concrete.*

Proof. $WCC.Concrete$ and $WCC.Hybrid$ share the same skeleton $WCC.Impl$ and differ only in the instances of Gather and RSD. Apply the pre-congruence of $\preceq_{PFS}$ for parallel composition (Theorem ??) to the child simulations 20 and 23, lifted along the $WCC.Impl$ context. $\square$

Lemma 18. *WCC.Spec simulates WCC.Hybrid.*

3.4 GBCA

Theorem 19. *GBCA.Spec simulates GBCA.Impl. No sub-protocol: direct proof.*

3.5 Gather

Theorem 20. *Gather.Spec simulates Gather.Concrete.*

Proof. By transitivity of probabilistic forward simulation (Lemma ??), chain 22 with 21 by transitivity. $\square$

Lemma 21. *Gather.Hybrid is simulated by Gather.Concrete.*

Proof. Gather.Concrete and Gather.Hybrid share the same skeleton Gather.Impl and differ only in the BRB instances. Apply the pre-congruence of $\preceq_{\text{PFS}}$ for parallel composition (Theorem ??) to the child simulation 26, lifted along the Gather.Impl context (one instance per leader). $\square$

Lemma 22. *Gather.Spec simulates Gather.Hybrid.*

3.6 RSD

Theorem 23. *RSD.Spec simulates RSD.Concrete.*

Proof. By transitivity of probabilistic forward simulation (Lemma ??), chain 25 with 24 by transitivity. $\square$

Lemma 24. *RSD.Hybrid is simulated by RSD.Concrete. Only BRB is substituted; AVSS.Spec stays fixed on both sides.*

Proof. RSD.Concrete and RSD.Hybrid share the same skeleton RSD.Impl and the same AVSS.Spec instances; only the BRB instances are substituted. Apply the pre-congruence of $\preceq_{\text{PFS}}$ for parallel composition (Theorem ??) to the child simulation 26, lifted along the RSD.Impl context (one BRB instance per leader). $\square$

Lemma 25. *RSD.Spec simulates RSD.Hybrid.*

3.7 BRB

Theorem 26. *BRB.Spec simulates BRB.Impl. No sub-protocol: direct proof.*

Bibliography

- [ABDY22] Ittai Abraham, Naama Ben-David, and Sravya Yandamuri. Efficient and adaptively secure asynchronous binary agreement via binding crusader agreement. In *41st ACM Symposium on Principles of Distributed Computing*, pages 381–391, 2022.
- [AFW25] Hagit Attiya, Itay Flam, and Jennifer L. Welch. Beyond canonical rounds: Communication abstractions for optimal byzantine resilience. *CoRR*, abs/2510.04310, 2025.
- [AW23] Hagit Attiya and Jennifer L. Welch. Multi-valued connected consensus: A new perspective on crusader agreement and adopt-commit. In *27th International Conference on Principles of Distributed Systems (OPODIS)*, 2023.
- [Bra87] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.
- [DSW22] Brijesh Dongol, Gerhard Schellhorn, and Heike Wehrheim. Weak Progressive Forward Simulation Is Necessary and Sufficient for Strong Observational Refinement. In Bartek Klin, Sławomir Lasota, and Anca Muscholl, editors, *33rd International Conference on Concurrency Theory (CONCUR 2022)*, volume 243 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 31:1–31:23, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [EMMS26] Constantin Enea, Rupak Majumdar, Harshit Jitendra Motwani, and V. R. Sathiyarayanan. Verifying almost-sure termination for randomized distributed algorithms. *Proc. ACM Program. Lang.*, 10(POPL):1412–1441, 2026.
- [FKT22] Luciano Freitas, Petr Kuznetsov, and Andrei Tonkikh. Distributed randomness from approximate agreement, 2022.
- [LT26] Nancy A Lynch and Mark R Tuttle. An introduction to input/output automata. *Form. Asp. Comput.*, January 2026. Just Accepted.
- [LV95] N. Lynch and F. Vaandrager. Forward and backward simulations. *Information and Computation*, 121(2):214–233, 1995.
- [SA21] Gilad Stern and Ittai Abraham. Living with asynchrony: the gather protocol. <https://decentralizedthoughts.github.io/2021-03-26-living-with-asynchrony-the-gather-protocol>, 2021.
- [SA24] Gilad Stern and Ittai Abraham. Gather with binding and verifiability. <https://decentralizedthoughts.github.io/2024-01-09-gather-with-binding-and-verifiability/>, 2024.

- [Seg95] Roberto Segala. A compositional trace-based semantics for probabilistic automata. In Insup Lee and Scott A. Smolka, editors, *CONCUR '95: Concurrency Theory*, pages 234–248, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.